



UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIAS E TECNOLOGIA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
SISTEMAS OPERACIONAIS

INFRAESTRUTURA VIRTUALIZADA HÍBRIDA

Implementação de Máquinas Virtuais e Microserviços com
KVM/QEMU e Docker

Jonathan Emerson B, da Silva

Boa Vista – RR
Junho de 2026

Sumário

1	Introdução	2
2	Desenvolvimento da Infraestrutura	3
2.1	Topologia da Infraestrutura	3
2.2	Configuração das Redes Virtuais	3
2.2.1	Rede NAT (default) — 192.168.122.0/24	3
2.2.2	Rede Interna (internal-net) — 10.0.10.0/24	3
2.3	Configuração da gateway-vm	3
2.3.1	/etc/network/interfaces	3
2.3.2	Roteamento e NAT	3
2.3.3	/etc/sysctl.conf	4
2.4	Configuração da app-vm	4
2.4.1	/etc/network/interfaces	4
2.5	Arquitetura de Microserviços (Docker Compose)	4
2.5.1	Estrutura de arquivos	4
2.5.2	app/Dockerfile	5
2.5.3	app/main.py	5
2.5.4	nginx/nginx.conf	5
2.5.5	docker-compose.yml	5
2.6	Pipeline de Validação	6
3	Topologia de Rede	7
3.1	Componentes e Endereçamento	7
3.2	Fluxo de Pacotes	7
3.3	Sub-redes	8
4	Matriz Comparativa: MV vs Containers	9
4.1	Primitivas de Isolamento do Kernel Linux	9
5	Sistemas Operacionais para Virtualização	10
5.1	Alpine Linux	10
5.2	Proxmox VE	10
5.3	Talos Linux	10
6	Relatório Quantitativo de Performance	12
6.1	Consumo de Recursos — Containers (docker stats)	12
6.2	Consumo de Recursos — Máquinas Virtuais	12
6.3	Análise Comparativa	12
6.4	Conclusões	12
7	Conclusão	13
8	Referências Bibliográficas	14

1. Introdução

A virtualização tornou-se uma das principais tecnologias utilizadas na construção de infraestruturas computacionais modernas, permitindo melhor aproveitamento de recursos físicos, isolamento de ambientes e maior flexibilidade operacional. Paralelamente, a popularização da arquitetura baseada em microsserviços impulsionou o uso de containers como alternativa leve e escalável para implantação de aplicações distribuídas.

Nesse contexto, a combinação entre máquinas virtuais e containers representa uma abordagem híbrida capaz de unir as vantagens de ambas as tecnologias. Enquanto as máquinas virtuais oferecem elevado nível de isolamento por meio da execução de sistemas operacionais independentes, os containers proporcionam inicialização rápida, menor consumo de recursos e facilidade de escalabilidade.

O presente trabalho apresenta a implementação de uma infraestrutura virtualizada híbrida utilizando o hipervisor KVM/QEMU para provisionamento de máquinas virtuais Alpine Linux e a plataforma Docker Compose para implantação de microsserviços. A solução desenvolvida contempla a configuração de redes virtuais, roteamento entre sub-redes, tradução de endereços de rede (NAT), balanceamento de carga com Nginx e persistência de dados utilizando Redis.

Além da implementação prática, são apresentados resultados experimentais relacionados ao consumo de recursos computacionais e uma análise comparativa entre máquinas virtuais e containers, permitindo avaliar suas características, vantagens e limitações em cenários de infraestrutura moderna.

2. Desenvolvimento da Infraestrutura

Este capítulo documenta os procedimentos de provisionamento da infraestrutura híbrida, incluindo configuração do hypervisor, redes virtuais e orquestração de containers.

2.1 Topologia da Infraestrutura

A infraestrutura é composta por duas camadas principais:

- **Camada de Hypervisor:** duas VMs Alpine Linux gerenciadas via KVM/QEMU (libvirt)
- **Camada de Containers:** três microsserviços orquestrados via Docker Compose no host Arch Linux

2.2 Configuração das Redes Virtuais

2.2.1 Rede NAT (default) — 192.168.122.0/24

```
virsh net-list --all
# Rede 'default' ativa - subnet 192.168.122.0/24
# gateway-vm: eth0 = 192.168.122.10 (DHCP)
```

2.2.2 Rede Interna (internal-net) — 10.0.10.0/24

Rede isolada sem DHCP. O bloco <dhcp> foi removido para evitar conflito com o IP estático da gateway-vm:

```
<network>
  <name>internal-net</name>
  <bridge name='virbr1' stp='on' delay='0' />
  <mac address='52:54:00:4d:26:fb' />
</network>
```

2.3 Configuração da gateway-vm

2.3.1 /etc/network/interfaces

```
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp

auto eth1
iface eth1 inet static
    address 10.0.10.1
    netmask 255.255.255.0
    post-up ip route add 10.0.10.0/24 dev eth1
```

2.3.2 Roteamento e NAT

```
# Habilitar IP forwarding
```

```
sysctl -w net.ipv4.ip_forward=1

# Regra MASQUERADE
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

# Persistir
rc-service iptables save
rc-update add iptables boot
```

2.3.3 */etc/sysctl.conf*

```
net.ipv4.ip_forward = 1
net.ipv4.conf.all.rp_filter = 0
net.ipv4.conf.eth0.rp_filter = 0
net.ipv4.conf.eth1.rp_filter = 0
```

2.4 Configuração da app-vm

Alpine instalado em /dev/sda modo sys. Parâmetros usados no setup-alpine:

Campo	Valor
Hostname	app-vm
Interface	eth0
IP	10.0.10.2 / 255.255.255.0
Gateway	10.0.10.1
DNS	8.8.8.8
Disco	/dev/sda — modo sys

2.4.1 */etc/network/interfaces*

```
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 10.0.10.2
    netmask 255.255.255.0
    gateway 10.0.10.1
```

2.5 Arquitetura de Microserviços (Docker Compose)

2.5.1 *Estrutura de arquivos*

```
infra-projeto/
|-- docker-compose.yml
|-- app/
|   |-- Dockerfile
|   '-- main.py
'-- nginx/
    '-- nginx.conf
```

2.5.2 *app/Dockerfile*

```
FROM python:3.11-alpine
WORKDIR /app
RUN pip install fastapi uvicorn redis
COPY main.py .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port",
    "8000"]
```

2.5.3 *app/main.py*

```
from fastapi import FastAPI
import redis, socket

app = FastAPI()
r = redis.Redis(host='redis', port=6379)

@app.get("/")
def root():
    hostname = socket.gethostname()
    r.incr("visitas")
    total = r.get("visitas").decode()
    return {"servico": "app", "container": hostname,
        "visitas_totais": total}

@app.get("/health")
def health():
    return {"status": "ok"}
```

2.5.4 *nginx/nginx.conf*

```
upstream app_cluster {
    server app_service:8000;
}
server {
    listen 80;
    location / {
        proxy_pass http://app_cluster;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

2.5.5 *docker-compose.yml*

```
services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
```

```
volumes:
  - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:
    ro
  depends_on: [app_service]

app_service:
  build: ./app
  expose: ["8000"]
  depends_on: [redis]

redis:
  image: redis:alpine
  volumes:
    - redis_data:/data

volumes:
  redis_data:

networks:
  default:
    name: infra_cluster_net
```

2.6 Pipeline de Validação

```
# 1. Inicializar
docker compose up -d --build

# 2. Escalar para 3 replicas
docker compose up --scale app_service=3 -d

# 3. Auditar consumo (cgroups)
docker stats

# 4. Inspeccionar rede
docker network inspect infra_cluster_net
```

3. Topologia de Rede

3.1 Componentes e Endereçamento

Camada	Componente	IP	Interface	Função
Host	jotarch	192.168.122.1	virbr0	Hypervisor / Docker Host
VM	gateway-vm	192.168.122.10	eth0	Gateway NAT
VM	gateway-vm	10.0.10.1	eth1	Gateway rede interna
VM	app-vm	10.0.10.2	eth0	Nó de aplicação
Container	nginx-1	172.18.0.4	infra_cluster	Balanceador
Container	app_service-1	172.18.0.3	infra_cluster	FastAPI
Container	app_service-2	172.18.0.5	infra_cluster	FastAPI
Container	app_service-3	172.18.0.6	infra_cluster	FastAPI
Container	redis-1	172.18.0.2	infra_cluster	Banco de dados

3.2 Fluxo de Pacotes

```

Cliente HTTP
  |
  v porta 80
[Host Arch - jotarch]
iptables DOCKER NAT
  |
  v
[nginx - 172.18.0.4] round-robin
  /      |      \
  v      v      v
[app-1]  [app-2]  [app-3]
.3      .5      .6
  \      |      /
  v      v      v
[redis - 172.18.0.2]
incr("visitas")

Rede interna das VMs:
[app-vm 10.0.10.2]
  |
  v rota default
[gateway-vm eth1 10.0.10.1]
  |
  MASQUERADE
  |
[eth0 192.168.122.10]
  |
Internet

```

3.3 Sub-redes

Sub-rede	Tipo	Bridge	DHCP	Uso
192.168.122.0/24	NAT	virbr0	Sim (libvirt)	Internet das VMs
10.0.10.0/24	Interna	virbr1	Não	Comunicação inter-VM
172.18.0.0/16	Docker	docker0	Sim (Docker)	Rede dos containers

4. Matriz Comparativa: MV vs Containers

Critério	Máquinas Virtuais (KVM)	Containers (Docker)
Isolamento	Completo: hardware emulado, kernel próprio por VM	Parcial: compartilham kernel do host, isolados via namespaces e cgroups
Boot time	15–45 segundos (kernel + init)	< 1 segundo (processo no kernel host)
RAM base	gateway-vm: ~85MB app-vm: ~60MB	nginx: ~12MB app: ~45MB redis: ~11MB
Portabilidade	Imagem qcow2 volumosa, dependente do hypervisor	Imagem OCI padrão, executável em qualquer host Docker
Overhead CPU	Alto: tradução de instruções (QEMU/VT-x)	Mínimo: syscalls nativas, sem interceptação
Segurança	Forte: falha no guest não afeta host	Moderada: escape via vulnerabilidade de kernel possível
Armazenamento	Disco virtual completo (qcow2): ~2GB	Layer-based (Union FS): imagem compartilhada
Escalabilidade	Lenta: nova VM demanda minutos	Rápida: -scale em segundos
Rede	Interface virtual (virtio-net), bridge libvirt	veth pairs, bridge docker0, DNS interno automático
Caso de uso	Kernel customizado, isolamento máximo	Microserviços stateless, CI/CD, alta densidade

Table 1: Comparativo técnico MV vs Containers

4.1 Primitivas de Isolamento do Kernel Linux

Primitiva	Máquinas Virtuais	Containers
Namespaces	VM inteira em namespace separado	net, pid, mnt, ipc, uts por container
Cgroups	Limitação via libvirt XML	<code>docker stats</code> / <code>-memory</code> / <code>-cpus</code>
Seccomp	Não aplicável (kernel separado)	Perfil seccomp padrão do Docker
Syscalls	Interceptadas pelo hypervisor (VMEXIT)	Diretas ao kernel host

5. Sistemas Operacionais para Virtualização

5.1 Alpine Linux

Sistema minimalista baseado em **musl libc** e **BusyBox**, projetado para segurança e eficiência em ambientes virtualizados e containers.

Características arquiteturais:

- Kernel Linux enxuto, sem módulos desnecessários
- Init baseado em **OpenRC** (mais leve que systemd)
- Gerenciador de pacotes **apk** rápido e leve
- Imagem base Docker de apenas **5MB** (Ubuntu ~80MB)
- **musl libc**: implementação mais segura e compacta que glibc

Métrica	Alpine Linux	Ubuntu 22.04
Tamanho ISO	~150 MB	~1.2 GB
RAM em idle (VM)	~60 MB	~350 MB
Tempo de boot	~3 s	~15 s
Imagem Docker	~5 MB	~80 MB

Uso neste projeto: Alpine foi utilizado como SO das VMs e como imagem base de todos os containers Docker.

5.2 Proxmox VE

Plataforma enterprise baseada em **Debian** que integra **KVM** (VMs) e **LXC** (containers) em uma única interface de gerenciamento.

- Kernel com patches ZFS e drivers integrados
- Suporte a *live migration* e clusters com HA
- SDN (Software Defined Networking) nativo
- Backup e snapshot com compressão e deduplicação
- Interface web completa para gerenciamento de VMs e containers

5.3 Talos Linux

SO **imutável** projetado para Kubernetes: sem shell, sem SSH, sem gerenciador de pacotes. Gerenciado inteiramente via API gRPC (**talosctl**).

- Sistema de arquivos raíz somente-leitura (squashfs)
- Atualizações atômicas com A/B partitions e rollback automático
- Configuração declarativa via YAML (GitOps-ready)
- Superfície de ataque mínima: sem **bash**, **curl** ou **wget** no sistema base

- Ideal para produção Kubernetes com requisitos de compliance

6. Relatório Quantitativo de Performance

6.1 Consumo de Recursos — Containers (`docker stats`)

Dados coletados com 3 réplicas de `app_service` em carga normal:

Container	CPU %	Mem	Mem %
nginx-1	0.00%	12.83 MiB	0.05%
app_service-1	0.10%	~45 MiB	0.19%
app_service-2	0.08%	~45 MiB	0.19%
app_service-3	0.09%	~45 MiB	0.19%
redis-1	0.28%	10.95 MiB	0.05%
Total	~0.55%	~159 MiB	~0.67%

6.2 Consumo de Recursos — Máquinas Virtuais

VM	RAM Alocada	RAM em Uso	vCPUs	Disco
gateway-vm	512 MB	~85 MB	1	~500 MB
app-vm	512 MB	~60 MB	1	~2 GB
Total	1024 MB	~145 MB	2	~2.5 GB

6.3 Análise Comparativa

Métrica	5 Containers	2 VMs Alpine	Vencedor
RAM total	~159 MB	~145 MB	Similar
RAM overhead	~23 MB	~145 MB	Containers
Inicialização	< 2 s	5–8 s	Containers
Escala horiz.	segundos	minutos	Containers
Isolamento	processo	kernel	VMs
Segurança	kernel compartilhado	kernel próprio	VMs

6.4 Conclusões

- **Containers são superiores em densidade:** 5 serviços com apenas 23MB de overhead base, contra 145MB das VMs só para manter kernel e init rodando.
- **VMs oferecem isolamento mais robusto:** kernel próprio impede que falhas em um guest afetem o host.
- **A abordagem híbrida é a mais eficiente:** VMs garantem o perímetro de segurança; containers maximizam a eficiência na camada de aplicação.
- **Escalabilidade:** `-scale app_service=3` adicionou 2 réplicas em menos de 2 segundos.

7. Conclusão

O projeto implementou com sucesso uma infraestrutura virtualizada híbrida de dois níveis:

- **Nível 1 — Hypervisor (KVM/QEMU):** duas VMs Alpine Linux com rede interna isolada (10.0.10.0/24) e gateway NAT funcional, com configurações persistidas após reboot.
- **Nível 2 — Containers (Docker Compose):** três microsserviços interdependentes (Nginx + FastAPI + Redis) com balanceamento de carga e escalabilidade dinâmica validada.

Principais desafios técnicos superados:

- Conflito entre o DHCP do libvirt e o IP estático da gateway-vm na rede interna
- Bridge `virbr1` com IP do host interferindo no roteamento inter-VM
- Configuração correta de `rp_filter` e `ip_forward` para NAT funcionar entre as sub-redes
- Persistência das configurações de rede e iptables após reinicialização das VMs

A infraestrutura está totalmente operacional e validada, com todos os comandos de auditoria (`docker stats`, `docker network inspect`) retornando dados consistentes com a topologia projetada.

8. Referências Bibliográficas

1. KVM. Kernel-based Virtual Machine Documentation. Disponível em: <https://www.kernel.org/doc/html/latest/virt/kvm/index.html>. Acesso em: junho de 2026.
2. Docker Inc. Docker Documentation. Disponível em: <https://docs.docker.com>. Acesso em: junho de 2026.
3. NGINX. Official Documentation. Disponível em: <https://nginx.org/en/docs/>. Acesso em: junho de 2026.
4. Redis Documentation. Disponível em: <https://redis.io/docs>. Acesso em: junho de 2026.
5. ALPINE LINUX. Alpine Linux Documentation. Disponível em: <https://docs.alpinelinux.org>. Acesso em: junho de 2026.
6. PROXMOX. Proxmox VE Administration Guide. Disponível em: https://pve.proxmox.com/wiki/Main_Page. Acesso em: junho de 2026.
7. TANENBAUM, Andrew S.; WETHERALL, David J. Redes de Computadores. 5. ed. São Paulo: Pearson, 2011.
8. SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Fundamentos de Sistemas Operacionais. 10. ed. Rio de Janeiro: LTC, 2018.